# AI-ORB V3 – MASTER-BUILDER-PROMPT

## WINDOWS ONE-CLICK INSTALLATION & STARTUP

### NEUE VERBINDLICHE ANFORDERUNG

AI-Orb soll am Ende als **vollständig installierbare Windows-Desktop-Anwendung** ausgeliefert werden.

Das Ziel ist:

> Der Benutzer lädt **eine einzige Installationsdatei** herunter, installiert AI-Orb und kann danach den Orb über ein Symbol im Windows-Startmenü starten.

Der Benutzer soll nach der Installation **keine technischen Abhängigkeiten manuell installieren oder konfigurieren müssen**.

---

# 41. ONE-CLICK WINDOWS INSTALLER

Erstelle einen professionellen Windows-Installer, der alle für AI-Orb benötigten Komponenten installiert.

Das Endergebnis soll beispielsweise sein:

```
AI-Orb-Setup.exe
```

Der Installer muss möglichst alle notwendigen Abhängigkeiten automatisch erkennen und installieren.

Der Benutzer soll nicht manuell installieren müssen:

- Python
- Node.js
- npm
- Electron Runtime
- benötigte Python-Pakete
- benötigte Node-Pakete
- Whisper-Abhängigkeiten
- Kokoro-Abhängigkeiten
- Audio-Abhängigkeiten
- Visual-C++-Runtime, sofern benötigt
- Ollama, sofern für die gewählte Architektur erforderlich
- weitere für AI-Orb erforderliche Runtime-Komponenten

Der Installer soll prüfen, welche Komponenten bereits vorhanden sind.

Sind sie bereits vorhanden:

→ nicht unnötig erneut installieren.

Fehlen sie:

→ automatisch installieren bzw. die benötigte Anwendungskomponente mitbringen.

---

# 42. SINGLE INSTALLER

Das bevorzugte Ziel ist:

```
AI-Orb-Setup.exe
```

Eine einzige Datei soll den kompletten Installationsprozess durchführen.

Der Benutzer soll nach Möglichkeit keine zusätzlichen Dateien von Hand herunterladen müssen.

Der Installer soll:

1. System prüfen
2. Windows-Version prüfen
3. Architektur prüfen
4. notwendige Voraussetzungen erkennen
5. fehlende Komponenten installieren
6. AI-Orb installieren
7. Konfiguration erstellen
8. benötigte lokale Modelle/Komponenten vorbereiten
9. Firewall-/Netzwerkanforderungen berücksichtigen
10. Startmenü-Eintrag erstellen
11. Desktop-Verknüpfung optional anbieten
12. Autostart optional anbieten
13. Installation testen
14. erfolgreichen Abschluss anzeigen

---

# 43. WINDOWS STARTMENÜ

Nach erfolgreicher Installation muss im Windows-Startmenü ein Eintrag erstellt werden:

```
AI-Orb
```

Der Eintrag muss ein eigenes AI-Orb-Icon besitzen.

Beispiel:

```
Windows Startmenü
    ↓
AI-Orb
    ↓
AI-Orb starten
```

Der Benutzer darf nicht zuerst:

- Terminal öffnen
- PowerShell öffnen
- Python starten
- npm ausführen
- Ollama manuell starten
- Backend manuell starten
- Electron manuell starten

müssen.

---

# 44. EIN KLICK ZUM STARTEN

Wenn der Benutzer im Startmenü auf:

```
AI-Orb
```

klickt, muss automatisch die komplette AI-Orb-Umgebung gestartet werden.

Ablauf:

```
Benutzer klickt auf AI-Orb
        ↓
AI-Orb Launcher
        ↓
System prüfen
        ↓
Backend starten
        ↓
Claude-Verbindung prüfen
        ↓
Ollama-Fallback prüfen/starten
        ↓
Memory starten
        ↓
Voice-System starten
        ↓
Whisper initialisieren
        ↓
```

```
Kokoro initialisieren
        ↓
Desktop-/Vision-System initialisieren
        ↓
Human-Experience-System starten
        ↓
Electron/UI starten
        ↓
Orb erscheint
        ↓
READY
```

Dieser gesamte Prozess muss automatisch erfolgen.

---

# 45. AI-ORB LAUNCHER

Erstelle einen eigenen robusten Launcher.

Der Launcher ist die zentrale Startkomponente der Windows-Version.

Der Launcher muss:

- Voraussetzungen prüfen
- benötigte Services starten
- Ports prüfen
- bereits laufende Prozesse erkennen
- Backend starten
- Ollama starten, falls erforderlich
- Voice-Komponenten starten
- UI starten
- Fehler erkennen
- Logs schreiben
- Prozesse sauber beenden können

Der Launcher darf keine sichtbare Kommandozeile benötigen.

Optional kann während des Starts ein kleiner Splash-/Loading-Screen angezeigt werden.

Beispiel:

```
AI-Orb wird gestartet...

✓ System
✓ Backend
✓ Claude
✓ Offline-KI
✓ Voice
```

```
✓ Memory
✓ Desktop
✓ Human Experience

AI-Orb bereit
```

---

# 46. KEINE MANUELLE ABHÄNGIGKEITSINSTALLATION

Nach der Installation soll der Benutzer nicht mit Fehlermeldungen wie:

```
python is not recognized
npm is not recognized
pip is not recognized
node is not recognized
module not found
ollama not found
```

konfrontiert werden.

Die Anwendung muss ihre Runtime entweder:

1. mitbringen,
2. während der Installation automatisch installieren,
3. oder eine bereits vorhandene kompatible Installation verwenden.

Die bevorzugte Lösung ist eine möglichst **self-contained Distribution**.

---

# 47. PYTHON

Wenn das Backend Python benötigt, soll der Windows-Build nicht voraussetzen, dass der Benutzer Python installiert hat.

Bevorzugte Lösung:

Eine eigene gebündelte Python-Runtime bzw. ein vollständig gebündeltes Python-Backend.

Das Backend soll so ausgeliefert werden, dass der Benutzer Python nicht kennen oder bedienen muss.

Beispiel:

```
AI-Orb
 └── runtime
      └── python
```

Der Launcher verwendet diese Runtime.

---

# 48. ELECTRON

Electron muss Bestandteil des fertigen Windows-Builds sein.

Der Benutzer muss Node.js oder Electron nicht separat installieren.

Der finale Build soll eine normale Windows-Desktop-Anwendung darstellen.

---

# 49. OLLAMA

Ollama ist der Offline-Fallback.

Prüfe beim Start:

```
Ist Ollama vorhanden?
        ↓
JA → Health Check
        ↓
NEIN → automatisch vorbereiten/installieren
```

Wenn Ollama als separat installierte Komponente technisch erforderlich ist, muss der Installer dies automatisch erledigen.

Der Benutzer darf Ollama nicht manuell über die Kommandozeile starten müssen.

Beim Start von AI-Orb:

```
AI-Orb Launcher
      ↓
Ollama verfügbar?
      ↓
falls erforderlich:
Ollama starten
      ↓
llama3.1:8b verfügbar?
```

```
      ↓
ja → READY
```

Falls das Modell noch nicht vorhanden ist, muss die Anwendung dies erkennen und dem Benutzer einen klaren Installations-/Downloadstatus anzeigen.

---

# 50. CLAUDE

Claude bleibt die primäre KI.

Beim Start:

```
Claude Connection Check
```

Wenn die konfigurierte Claude-Verbindung funktioniert:

```
PRIMARY AI: CLAUDE
```

Wenn Claude nicht erreichbar ist:

```
CLAUDE UNAVAILABLE
↓
OFFLINE MODE
↓
OLLAMA
```

Der Benutzer muss jederzeit erkennen können, welcher Modus aktiv ist.

---

# 51. WHISPER UND KOKORO

Whisper und Kokoro müssen ebenfalls Bestandteil des finalen Installationskonzepts sein.

Der Benutzer soll keine Python-Pakete manuell installieren.

Der Installer bzw. die Anwendung muss:

- benötigte Runtime
- Bibliotheken
- Modelle
- Audio-Abhängigkeiten

automatisch vorbereiten.

Beim Start:

```
Voice System
 ↓
Whisper
 ↓
Kokoro
 ↓
Audio Device Check
 ↓
READY
```

---

# 52. INSTALLATIONSVERZEICHNIS

Verwende ein sauberes Windows-Verzeichnis.

Beispielsweise:

```
C:\Program Files\AI-Orb
```

Benutzerdaten dürfen nicht direkt dort gespeichert werden.

Verwende für Benutzer-/Konfigurationsdaten geeignete Windows-Verzeichnisse.

Beispielsweise:

```
%APPDATA%\AI-Orb
```

oder

```
%LOCALAPPDATA%\AI-Orb
```

Dort können gespeichert werden:

- Konfiguration
- Logs
- Memory
- User Preferences
- Positionierungsdaten
- Heatmap
- lokale Datenbanken
- Cache
- Modelle, sofern sinnvoll

- temporäre Dateien

---

# 53. STARTMENÜ-ICON

Der Installer muss ein korrektes Windows-Icon installieren.

Benötigte Formate/Größen müssen für Windows korrekt bereitgestellt werden.

Das Icon soll für:

- Installer
- Startmenü
- Anwendung
- Taskleiste
- optional Desktop

verwendet werden.

Kein generisches Electron-Icon.

---

# 54. DESKTOP-VERKNÜPFUNG

Während der Installation kann optional angeboten werden:

```
☑ Desktop-Verknüpfung erstellen
```

Standardmäßig sollte die Startmenü-Verknüpfung immer erstellt werden.

---

# 55. AUTOSTART

Optional:

```
□ AI-Orb beim Windows-Start automatisch starten
```

Wenn aktiviert:

Windows startet → AI-Orb Launcher → Abhängigkeiten → Orb

Der Benutzer muss diese Funktion jederzeit deaktivieren können.

---

# 56. UPDATE-SYSTEM

Der Installer muss später aktualisierbar sein.

Ein Update darf nicht automatisch:

- Memory löschen
- User Preferences löschen
- Projekte löschen
- Knowledge Graph löschen
- Positionierungsdaten löschen

Programmdateien und Benutzerdaten müssen getrennt sein.

---

# 57. UNINSTALLER

Der Installer muss einen Windows-Uninstaller bereitstellen.

Der Benutzer kann AI-Orb über:

```
Windows Einstellungen
→ Apps
→ Installierte Apps
→ AI-Orb
→ Deinstallieren
```

entfernen.

Der Uninstaller muss klar unterscheiden zwischen:

```
Programm entfernen
```

und

```
Benutzerdaten löschen
```

Benutzerdaten sollen nicht ohne eindeutige Zustimmung gelöscht werden.

---

# 58. CRASH RECOVERY

Wenn ein AI-Orb-Prozess abstürzt:

```
Process crashed
 ↓
Launcher erkennt Fehler
 ↓
Logging
 ↓
Recovery
 ↓
Neustart des betroffenen Dienstes
```

Nicht sofort den gesamten Orb beenden, wenn nur ein optionales Modul ausgefallen ist.

Beispiel:

Wenn Kokoro ausfällt:

→ Textantwort weiterhin anzeigen.

Wenn Ollama ausfällt, aber Claude funktioniert:

→ Claude weiter verwenden.

Wenn Claude ausfällt:

→ Ollama verwenden.

Wenn Vision ausfällt:

→ Orb weiter betreiben.

---

# 59. HEALTH CHECK

Beim Start muss jeder wichtige Dienst überprüft werden.

Beispiel:

```
AI-ORB STARTUP

[✓] Windows
[✓] Runtime
[✓] Backend
[✓] ProviderManager
[✓] Claude
[✓] Ollama
[✓] Whisper
[✓] Kokoro
```

```
[✓] Memory
[✓] Desktop
[✓] Human Experience
[✓] Orb UI

STATUS: READY
```

Fehler müssen verständlich erklärt werden.

---

# 60. INSTALLER-TECHNOLOGIE

Wähle eine professionelle Windows-Installer-Technologie.

Geeignete Optionen sind beispielsweise:

- NSIS
- WiX
- electron-builder

Wähle die Technologie, die am besten zur bestehenden Electron-Architektur passt.

Wichtig ist nicht die konkrete Installer-Technologie, sondern das Endergebnis:

```
Eine einzige Installationsdatei
        ↓
Installation
        ↓
Startmenü
        ↓
AI-Orb
        ↓
Ein Klick
        ↓
vollständige Anwendung startet
```

---

# 61. FINALER BENUTZERWORKFLOW

Der komplette Benutzerworkflow muss so einfach wie möglich sein:

**SCHRITT 1**

Benutzer lädt herunter:

```
AI-Orb-Setup.exe
```

### SCHRITT 2

Benutzer startet die Datei.

### SCHRITT 3

Installer installiert automatisch:

- AI-Orb
- Runtime
- Backend
- notwendige Bibliotheken
- Voice-Komponenten
- notwendige lokale Komponenten
- Ollama bzw. benötigte Offline-Komponenten
- Startmenü-Integration

### SCHRITT 4

Installer erstellt:

```
Startmenü
└── AI-Orb
```

### SCHRITT 5

Benutzer klickt auf:

```
AI-Orb
```

### SCHRITT 6

Launcher startet automatisch:

```
Backend
Claude
Ollama-Fallback
Whisper
Kokoro
Memory
Desktop-System
Human Experience
Orb UI
```

**SCHRITT 7**

Der Orb erscheint.

**SCHRITT 8**

Der Benutzer kann sofort sprechen:

> "Hallo Orb."

Keine Kommandozeile. Keine manuelle Dependency-Installation. Kein `pip install`. Kein `npm install`. Kein `python main.py`. Kein manuelles Starten von Ollama. Kein manuelles Starten des Backends.

---

# 62. ENTWICKLUNGSREGEL FÜR DEN BUILDER

Die Windows-Distribution darf nicht erst am Ende als nachträgliches Feature behandelt werden.

Die gesamte Architektur muss von Anfang an so gebaut werden, dass sie sauber paketiert werden kann.

Jede neue Komponente muss deshalb beantworten können:

1. Wie wird sie installiert?
2. Welche Runtime benötigt sie?
3. Welche Dateien benötigt sie?
4. Wie wird sie gestartet?
5. Wie wird geprüft, ob sie funktioniert?
6. Wie wird sie beendet?
7. Wie wird sie aktualisiert?
8. Wie wird sie deinstalliert?
9. Wie wird ein Fehler behandelt?

---

# 63. ABSOLUTES ENDERGEBNIS

Das fertige Projekt soll für den normalen Benutzer aussehen wie eine normale professionelle Windows-Anwendung.

Der Benutzer soll nichts über:

- Python
- Node
- Electron
- Pip
- npm
- virtuelle Umgebungen

- Backend-Prozesse
- Ports
- Ollama
- lokale Modelle
- Abhängigkeiten

wissen müssen.

Die technische Komplexität bleibt im Hintergrund.

Das gewünschte Erlebnis ist:

```
DOWNLOAD
   ↓
AI-Orb-Setup.exe
   ↓
INSTALLIEREN
   ↓
Startmenü → AI-Orb
   ↓
KLICK
   ↓
AI-Orb startet
   ↓
Claude
Whisper
Kokoro
Ollama
Memory
Human Experience
Desktop Intelligence
Orb
   ↓
READY
```

## FINALE PRODUKTANFORDERUNG

AI-Orb soll sich für den Benutzer wie **eine einzige installierte Anwendung** anfühlen, obwohl intern mehrere Komponenten und Dienste notwendig sein können.

Die gesamte technische Komplexität muss durch Installer, Launcher, Service-Management und automatische Health Checks verborgen werden.

**Ein Download.**
**Eine Installation.**
**Ein Startmenü-Icon.**
**Ein Klick.**
**AI-Orb läuft.**